

Factors and Data Frames

2

Learning Objectives

A systematic study of this chapter will help the reader understand the following concepts:

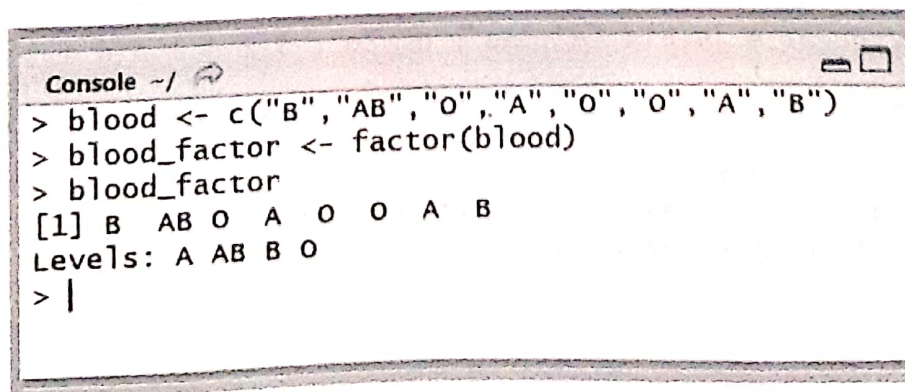
- Factor levels, summary, ordered factors
- Data frame creation, subsetting, sorting

2.1 INTRODUCTION TO FACTORS

There are three kinds of variables in R programming—categorical, ordinal, interval. In statistical modelling, categorical variables are those that can take on a limited number of different values. Since R is a statistical programming language, there exists a specific data structure for this called *factor*.

A *categorical variable* is one that has two or more categories, but there is no intrinsic ordering to the categories. For example, gender is one categorical variable with male and female being the possible values. A person belonging to a party, may either be a Democratic or a Republican. One's blood type can be A, B, AB, or O. In all these cases, there is no intrinsic order to the possible values that the variable can take. An *ordinal variable* is a categorical variable, but there is an intrinsic order present for the values it can take. The economic status may be low, medium, or high and grades in college are S, A, B, C, D, E, F. So these are some examples of variables where intrinsic ordering is present. Another type of variable present is the interval variable. These variables are defined by specific ranges such as \$10–\$15, age in the range of 20–25, height ranging from 5 feet 5 inches to 5 feet 9 inches; hence these are all interval variables.

A categorical variable is used as a part of the definition for the factors involved in programming. A *factor* is a statistical data type that stores categorical variables, which is used in statistical modelling. Factors are stored as a vector of integer values with a set of character values. This section discusses programming categorical variables or factors using R. To convert this vector to a factor, the `factor()` function can be used as shown in Fig. 2.1.

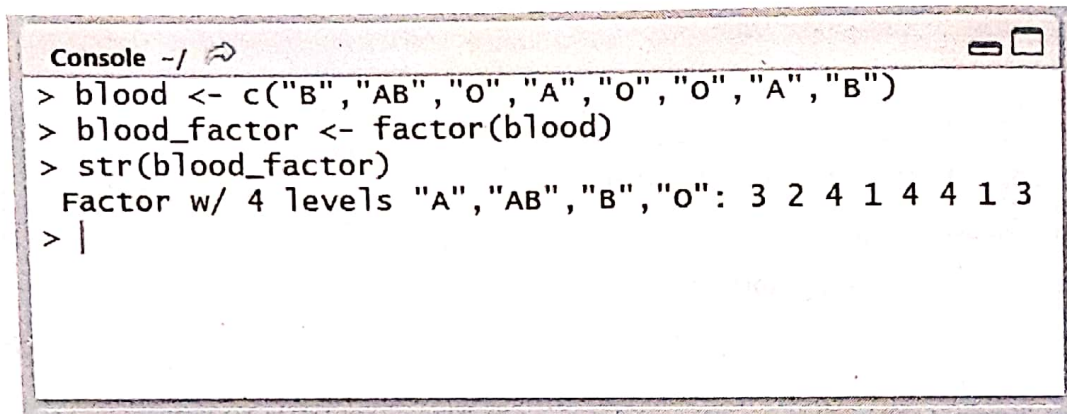


```
Console ~/
> blood <- c("B", "AB", "O", "A", "O", "O", "A", "B")
> blood_factor <- factor(blood)
> blood_factor
[1] B AB O A O O A B
Levels: A AB B O
```

Fig. 2.1 `factor()` function to convert vector to factor

R performs two functions when the *factor()* function is called on a character vector:

1. First, it scans through the vector to check the availability of different categories, In Fig. 2.1, R performs a check for categories of blood groups among "A", "AB", "B", and "O".
2. Further it sorts the levels alphabetically as can be observed from Fig. 2.2.



```

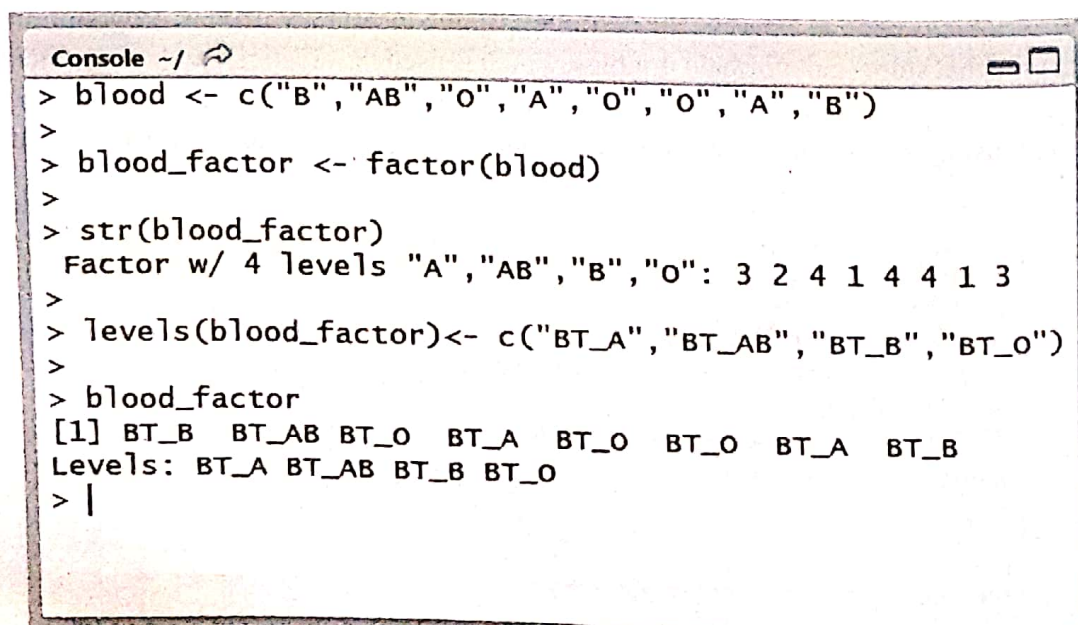
Console ~/
> blood <- c("B", "AB", "O", "A", "O", "O", "A", "B")
> blood_factor <- factor(blood)
> str(blood_factor)
Factor w/ 4 levels "A","AB","B","O": 3 2 4 1 4 4 1 3
> |
  
```

Fig. 2.2 Displaying different categories of blood groups

Here, it converts the character vector, *blood* in Fig. 2.1, to a vector of integer values. These integers correspond to a set of character values to use when the factor is displayed. Inspecting the structure in Fig. 2.2, it can be observed that a factor with a level of four is considered. "A" is encoded as 1, because it is the first level, "AB" is encoded as 2, "B" as 3, and "O" as 4 as shown.

2.1.1 Factor Levels

Factors are integer vectors where each integer corresponds to a category or a level. When the dataset is received, it can be noticed that it contains factors with specific factor levels. However, sometimes the names for these levels need to be changed for clarity or other reasons. R allows programmers to perform this task using the function *levels()* as shown in Fig. 2.3. In case a different ordering of the levels is required, the argument can be specified in the factor function.



```

Console ~/
> blood <- c("B", "AB", "O", "A", "O", "O", "A", "B")
>
> blood_factor <- factor(blood)
>
> str(blood_factor)
Factor w/ 4 levels "A","AB","B","O": 3 2 4 1 4 4 1 3
>
> levels(blood_factor) <- c("BT_A", "BT_AB", "BT_B", "BT_O")
>
> blood_factor
[1] BT_B BT_AB BT_O BT_A BT_O BT_O BT_A BT_B
Levels: BT_A BT_AB BT_B BT_O
> |
  
```

Fig. 2.3 Different ordering of levels

To change the order of the levels, it is possible to manually specify the level names, instead of letting R choose them. To display the blood types as 'BT_A', 'BT_AB', 'BT_B', and 'BT_O', we can use the *levels()* function. Similar to the *names()* function to name vectors, a vector can be passed to the level *blood_factor*. We can specify the category names or levels by using the *labels* argument in the *factor()* function. It is significant to follow the order of the factor levels: first A, followed by AB, then B, and finally O. It can be observed from Fig. 2.4 that erroneous results are obtained if the order is incorrect.

```

Console -/
> blood <- c("B", "AB", "O", "A", "O", "O", "A", "B")
> blood
[1] "B" "AB" "O" "A" "O" "O" "A" "B"
>
> blood_factor <- factor(blood)
>
> levels(blood_factor) <- c("BT_A", "BT_AB", "BT_B", "BT_O")
>
> blood_factor
[1] BT_B BT_AB BT_O BT_A BT_O BT_O BT_A BT_B
Levels: BT_A BT_AB BT_B BT_O
>
> factor(blood, labels=c("BT_A", "BT_A", "BT_AB", "BT_B", "BT_O"))
Error in factor(blood, labels = c("BT_A", "BT_A", "BT_AB", "BT_B", "BT_O")) :
  invalid 'labels'; length 5 should be 1 or 4
> |

```

Fig. 2.4 Error message when the order is changed

To solve the scenario in Fig. 2.4, a combination of 'levels' and 'labels' are manually specified as arguments when creating a factor. With levels, order can be specified, while with labels, a new name for the categories can be specified as shown in Fig. 2.5.

```

Console -/
> blood <- c("B", "AB", "O", "A", "O", "O", "A", "B")
> blood
[1] "B" "AB" "O" "A" "O" "O" "A" "B"
>
> blood_factor <- factor(blood)
>
> factor(blood,
+       levels=c("O", "A", "B", "AB"),
+       labels=c("BT_O", "BT_A", "BT_B", "BT_AB"))
[1] BT_B BT_AB BT_O BT_A BT_O BT_O BT_A BT_B
Levels: BT_O BT_A BT_B BT_AB
> |

```

Fig. 2.5 Naming the categories using labels

2.1.2 Summarizing a Factor

Summary is a generic function used to produce summaries of the results. This will give a quick overview of the contents of a variable. For instance, in Fig. 2.6, summary of the *blood_factor* object shows the categories of blood groups A, AB, B, and O and the summary of the object *blood* shows us the number of elements in the vector (which is 8), the class, and the mode of that vector.

```

Console ~/
> summary(blood_factor)
 A AB  B  O
 2  1  2  3
>
> summary(blood)
      Length      class      Mode
      8 character character
> |
  
```

Fig. 2.6 Summarizing the result

2.1.3 Ordered Factors

There are examples where a natural ordering exists; sometimes programmers have to deal with factors that do have a natural ordering among their categories. In this case, programmers have to make sure that this information is passed to R. In the domain of categorical variables, there exists a difference between nominal categorical variables and ordinal categorical variables. A nominal categorical variable, as mentioned earlier, has no implied order.

Example 2.1 Write a command in R console to create a *tshirt_factor*, which is ordered with levels 'S', 'M', and 'L'. Is it possible to identify from the examples discussed earlier, if blood type 'O' is greater or less than blood type 'A'?

Solution:

From Fig. 2.7 we can see that it is difficult to identify whether blood type "O" is greater or less than blood type "A".

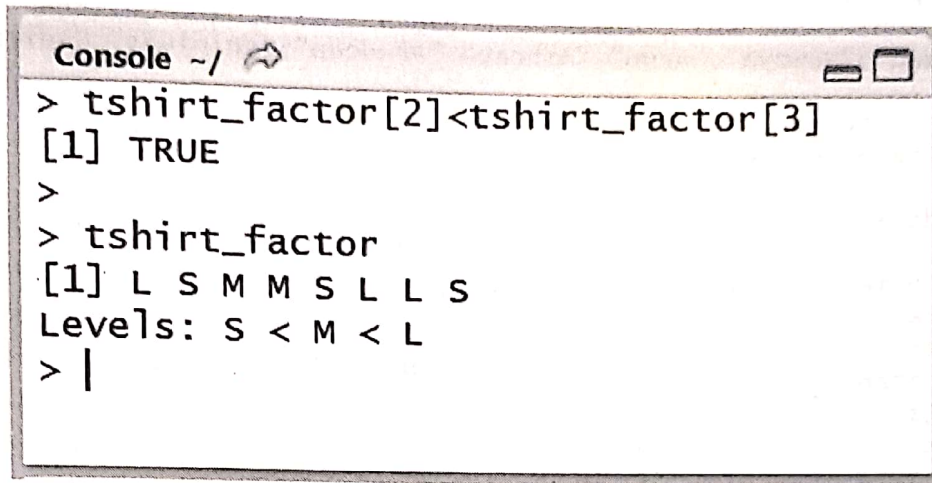
```

Console ~/
> blood <- c("B", "AB", "O", "A", "O", "O", "A", "B")
>
> blood_factor <- factor(blood)
>
> tshirt <- c("L", "S", "M", "M", "S", "L", "L", "S")
> tshirt_factor <- factor(tshirt, ordered = TRUE, levels = c("S", "M", "L"))
>
> tshirt_factor
[1] L S M M S L L S
Levels: S < M < L
> |
  
```

Fig. 2.7 Nominal categorical variable

2.1.4 Comparing Ordered Factors

It is possible to compare ordered factors in R. This is explained in Fig. 2.8.



```

Console ~/
> tshirt_factor[2]<tshirt_factor[3]
[1] TRUE
>
> tshirt_factor
[1] L S M M S L L S
Levels: S < M < L
> |
  
```

Fig. 2.8 Comparing ordered factors

In this example, it is specified that L is indeed greater than M. R provides a way to impose this kind of order on a factor, thus making it an ordered factor. This is achieved by the *factor()* function, where the argument *ordered* is set to TRUE and the levels are specified in ascending order.

2.2 INTRODUCTION TO DATA FRAME

Data frames are most significant and a widely accepted concept in R programming. R is a statistical programming language, where datasets are used. Data sets typically comprise observations, or instances, that have some variables associated with them. For example, consider a data set of five people, where each person is an instance and the properties about these people, such as their names, ages, and whether they have children, are the variables. A matrix cannot be used to store such information in R, as name would be a character and age would be numeric and these will not fit in a matrix. Data frame is a fundamental data structure to store data sets; it is similar to a spreadsheet that has rows and columns, where each column can be a different vector. Hence it is not necessary to have a data frame of all numbers; it can include numbers, characters, text, and so on.

2.2.1 Creating a Data Frame

This section discusses the process of creation of a data frame by considering an example consisting of employee details of a company.

Initially three individual vectors need to be created consisting of the following details: IDs numbered 1 through 15, name, and age. These three vectors are three independent equal length vectors, in which two of them are numerical and one is a character or text.

In the next step, data needs to be converted into a spreadsheet or data frame. A data frame is created using the function *data.frame()* by using a variable *data* as shown in Fig. 2.9. The parameters are the vectors created earlier, which are passed to the data frame as follows:

```
data.frame(ID, age, name)
```

The contents of the data frame can be displayed by printing the contents of the variable *data* as shown in Fig. 2.9, which looks like a spreadsheet.

```

Console - /
> id<-1:15
> age<-c(18,19,29,18,20,22,22,25,18,23,23,22,20,18,19)
> name<-c("Ishanni","Shanthi","Chetan","Nagayya","Nikitha","Chaitra","Vidyaraj",
"Deepak","Eshaan","Sowmya","Arun","Ashray","Pradeep","srinivas","Adithi")
>
> #Create a data frame
> data<-data.frame(id,age,name)
>
> #Print the data
> data
  id age  name
1  1  18 Ishanni
2  2  19 Shanthi
3  3  29  Chetan
4  4  18 Nagayya
5  5  20 Nikitha
6  6  22 Chaitra
7  7  22 Vidyaraj
8  8  25 Deepak
9  9  18 Eshaan
10 10  23 Sowmya
11 11  23  Arun
12 12  22 Ashray
13 13  20 Pradeep
14 14  18 srinivas
15 15  19 Adithi

```

Fig. 2.9 Creating a data frame

The number of rows and columns in a data frame can be computed using the functions `nrow()` and `ncol()` respectively. In order to print both the rows and columns, the function `dim()` can be used, which returns the dimension of the object as shown in Fig. 2.10.

```

Console - /
> #Display the number of Rows
> nrow(data)
[1] 15
>
> #Display the number of columns
> ncol(data)
[1] 3
>
> #Display both number of rows and columns
> dim(data)
[1] 15 3
>

```

Fig. 2.10 Displaying the number of rows and columns

The number of rows and columns of a data frame can be visualized in an environment window as shown in Fig. 2.11.

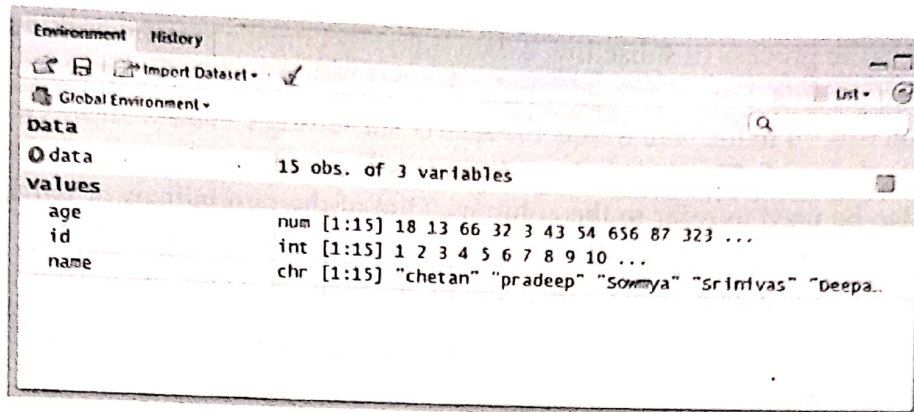


Fig. 2.11 View of data object in environment window

During visualizing, the big pieces of information which may contain thousands/millions of rows, very top or just the very bottom few rows can be seen, especially while importing a database from a website or from a spread sheet which generally contains a huge amount of information. To handle this scenario the function `head(data frame variable name)` can be used to show the first 6 rows of the data frame. The opposite of that function `tail(data frame variable name)` can be used to display the last 6 rows in the data frame. This can be seen in Fig. 2.12.

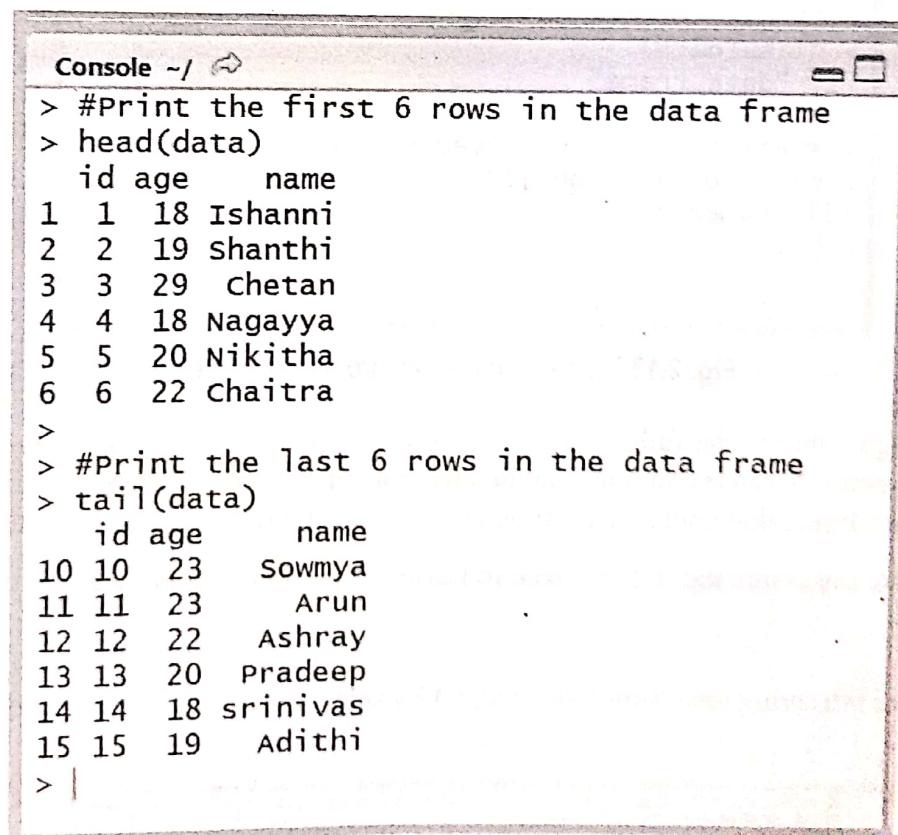


Fig. 2.12 Use of head and tail function in R

2.3 SUBSETTING OF DATA FRAMES

The data frame combines the features of matrices and lists. To subset a data frame from both matrices and lists, the following syntax can be used; Single brackets '[' are used for subsetting matrices and double brackets '[' and the dollar sign '\$' are used to select list elements.

Figure 2.13 illustrates the process of subsetting a data frame by considering the previous data set in Fig. 2.12 containing 15 employee records. For selecting a single element from the data frame by considering the age of Srinivas, who is on row 14 in the data frame, the matrix subsetting syntax with single brackets containing two indices inside can be used. The row index 14, comes first, and the column index 2, comes second. Column names can also be used to refer to the columns. One of the two indices or names can be omitted to end an entire row or an entire column. If all information on Srinivas is required, then the command to be used is `data[14,]`. The result is a data frame with a single observation (since the data are of different type

```

Console -/
> #Age of srinivas
> data[14,2]
[1] 18
>
> data[14,"age"]
[1] 18
>
> data[14,]
   id age  name
14 14  18 srinivas
>
> #class of data object
> class(data)
[1] "data.frame"
>
> #Data type of retrieved column
> class(data[, "age"])
[1] "numeric"
> |

```

Fig. 2.13 Selecting the data from data frame

To get the entire age column, the following command can be used: `data[, "age"]` as shown in Fig. 2.14. Here, the result is a vector, because columns contain elements of the same type. Subsetting the data frame to end up with a sub-data frame that contains multiple observations works in the same pattern.

Example 2.2 Write the command in R console to retrieve the name and id information from the 2nd and 3rd rows.

Solution:

Figure 2.14 illustrates retrieving data from rows 2 and 3 by selecting the name and id information of Chetan and Shanthi.

```

Console -/
> #Retrieve the id information with name for
  chetan and shanthi
> data[c(2,3),c("id", "name")]
   id  name
2  2 shanthi
3  3  chetan
> |

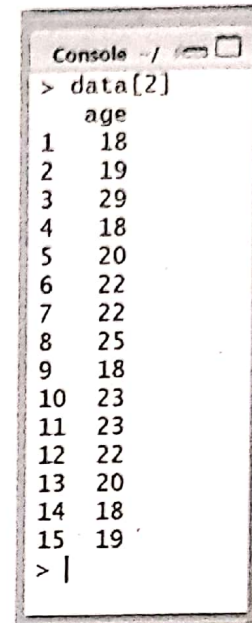
```

Fig. 2.14 Retrieving information from a data frame

All these figures illustrate that subsetting of data frames can be performed in a similar manner to subsetting of matrices. The only difference is when a single index is specified inside 'data'. In the case of matrix, R would go through each column from left to right to find the index specified. In the case of data frame, we will end up with a new data frame that contains only the column specified.

The command in Fig. 2.15 gives the age column as a data frame and not a vector. A new data frame is created from the existing one. Subsetting data frames with the list syntax will clarify the question of why this results in a data frame and not a vector.

When a data frame is treated as a list containing vectors of the same length, the list syntax can be used to select elements. For example, the command `data$age` returns the age vector inside the data frame as shown in Fig. 2.16. Further, the double brackets notation with a name or with an index can also be used. In all cases, the result is a vector as illustrated in the figure.

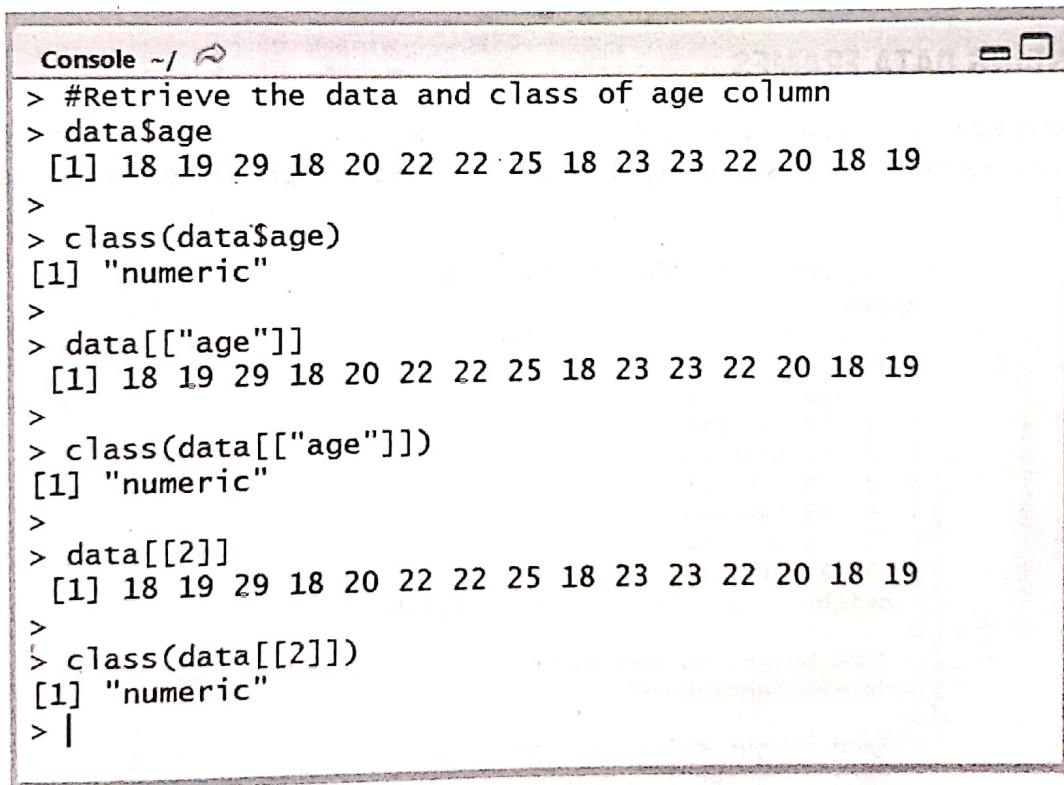


```

Console ~/
> data[2]
  age
1  18
2  19
3  29
4  18
5  20
6  22
7  22
8  25
9  18
10 23
11 23
12 22
13 20
14 18
15 19
> |

```

Fig. 2.15 Creating a new data frame from an existing one



```

Console ~/
> #Retrieve the data and class of age column
> data$age
[1] 18 19 29 18 20 22 22 25 18 23 23 22 20 18 19
>
> class(data$age)
[1] "numeric"
>
> data[["age"]]
[1] 18 19 29 18 20 22 22 25 18 23 23 22 20 18 19
>
> class(data[["age"]])
[1] "numeric"
>
> data[[2]]
[1] 18 19 29 18 20 22 22 25 18 23 23 22 20 18 19
>
> class(data[[2]])
[1] "numeric"
> |

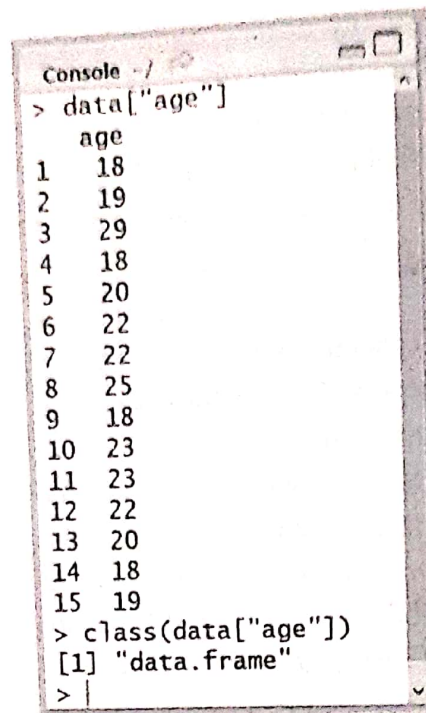
```

Fig. 2.16 Retrieving the data from data frame

Example 2.3 Write the command in R console to create a new data frame containing the 'age' parameter from the existing data frame. Check if the result is a data frame or not.

Solution:

Single brackets can also be used to subset lists; this generates a new list containing only the specified elements. The result is still a data frame, which is a list, but containing only the "age" element as shown in Fig. 2.17. This enumerates why this command gave a data frame in Fig. 2.15 instead of vector.



```

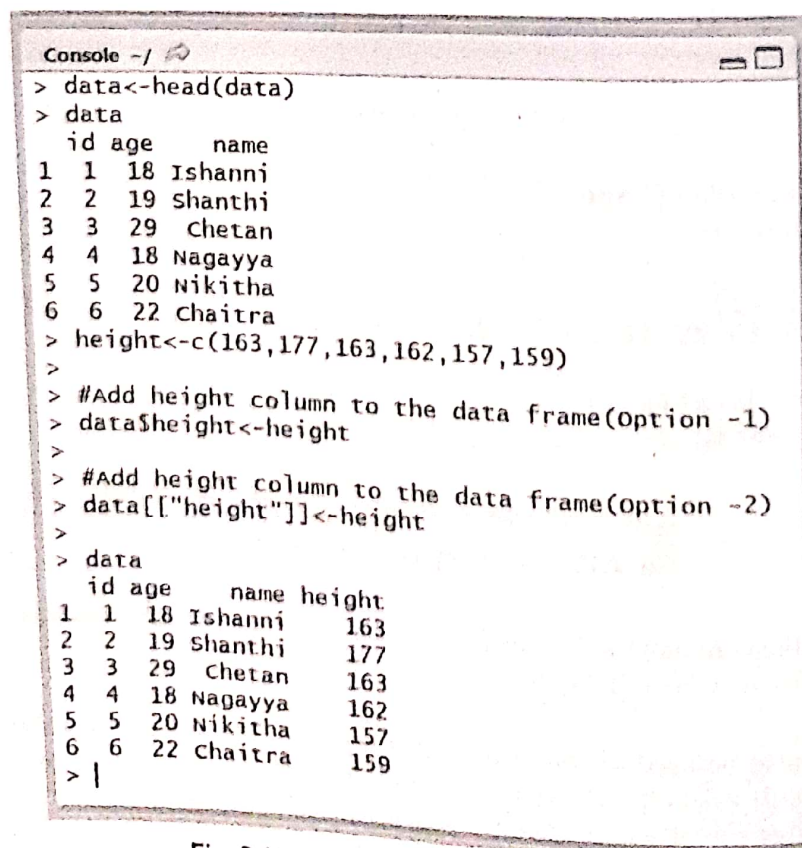
Console - /
> data["age"]
  age
1  18
2  19
3  29
4  18
5  20
6  22
7  22
8  25
9  18
10 23
11 23
12 22
13 20
14 18
15 19
> class(data["age"])
[1] "data.frame"
>

```

Fig. 2.17 Creating a new list from existing list

2.4 EXTENDING DATA FRAMES

To add a column, new variable, new row, or new observation to the existing data frame, the dollar sign (\$) and the double square brackets can be used. In Fig. 2.18, a vector named 'height' is created and is added to the data frame using \$ and [].



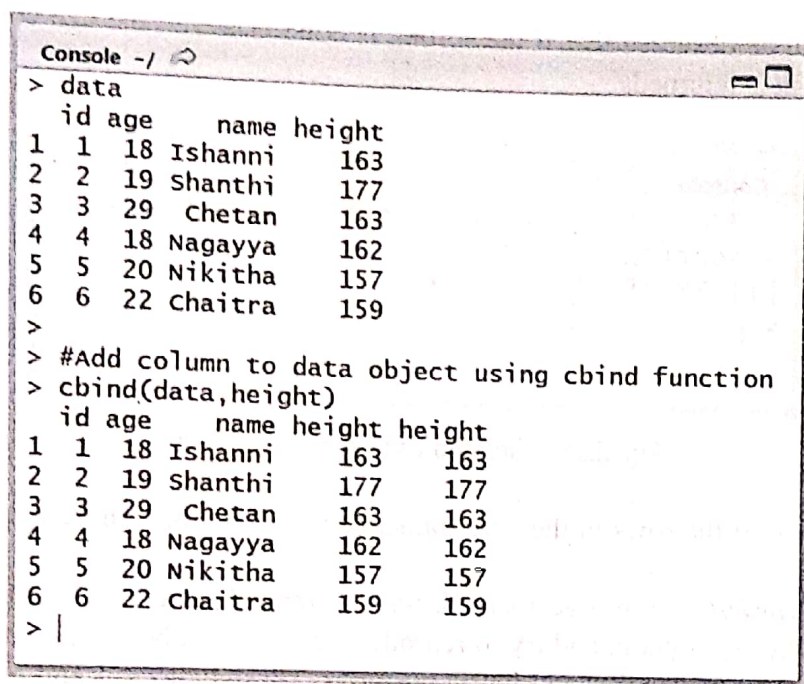
```

Console - /
> data<-head(data)
> data
  id age  name
1  1  18 Ishanni
2  2  19 Shanthi
3  3  29 Chetan
4  4  18 Nagayya
5  5  20 Nikitha
6  6  22 Chaitra
> height<-c(163,177,163,162,157,159)
>
> #Add height column to the data frame(option -1)
> data$height<-height
>
> #Add height column to the data frame(option -2)
> data[["height"]]<-height
>
> data
  id age  name height
1  1  18 Ishanni  163
2  2  19 Shanthi  177
3  3  29 Chetan  163
4  4  18 Nagayya  162
5  5  20 Nikitha  157
6  6  22 Chaitra  159
>

```

Fig. 2.18 Adding a column to the data set

The `cbind()` function can also be used to build and extend matrices. It works similar to the data frames shown in Fig. 2.18. If `cbind()` works, then `rbind()` will work fine as well. Indeed, `rbind()` is used to add new rows to observations. To add the information of another employee named Karan to the data frame, simply creating a vector with the `id`, `name`, `age`, and `height`, won't work, because a vector can't contain elements of different types. A new data frame needs to be created containing only a single observation, and that needs to be added to the existing data frame using `rbind()`. The usage of `cbind()` and `rbind()` are shown in Figs 2.19 and 2.20.

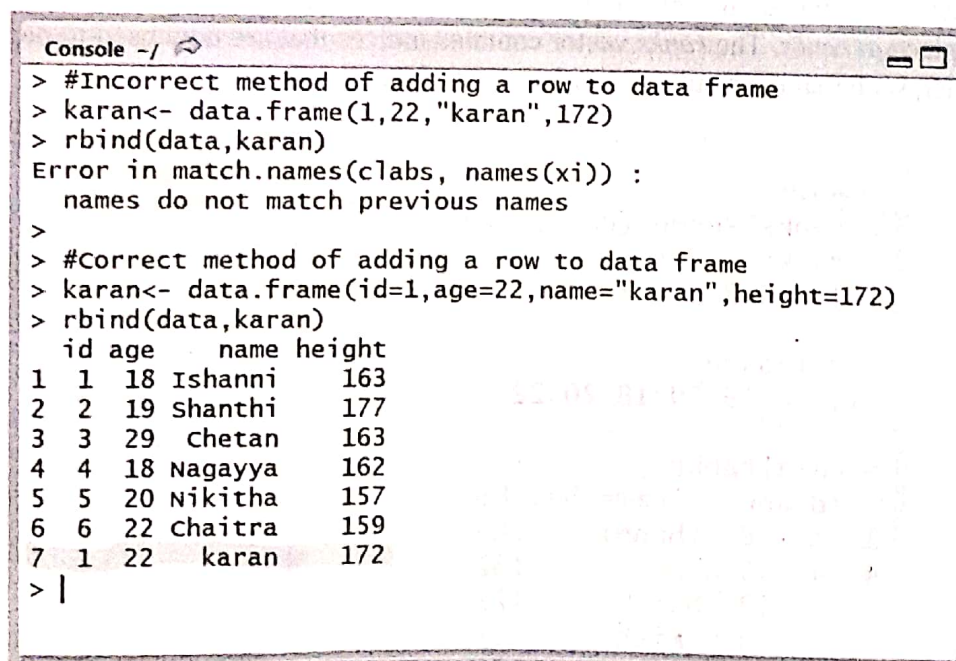


```

Console -/ 
> data
  id age  name height
1  1  18 Ishanni  163
2  2  19 Shanthi  177
3  3  29  Chetan  163
4  4  18 Nagayya  162
5  5  20 Nikitha  157
6  6  22 Chaitra  159
> 
> #Add column to data object using cbind function
> cbind(data,height)
  id age  name height height
1  1  18 Ishanni  163    163
2  2  19 Shanthi  177    177
3  3  29  Chetan  163    163
4  4  18 Nagayya  162    162
5  5  20 Nikitha  157    157
6  6  22 Chaitra  159    159
> |

```

Fig. 2.19 Adding a column to the data set using `cbind()`



```

Console -/ 
> #Incorrect method of adding a row to data frame
> karan<- data.frame(1,22,"karan",172)
> rbind(data,karan)
Error in match.names(clabs, names(xi)) :
  names do not match previous names
> 
> #Correct method of adding a row to data frame
> karan<- data.frame(id=1,age=22,name="karan",height=172)
> rbind(data,karan)
  id age  name height
1  1  18 Ishanni  163
2  2  19 Shanthi  177
3  3  29  Chetan  163
4  4  18 Nagayya  162
5  5  20 Nikitha  157
6  6  22 Chaitra  159
7  1  22  karan   172
> |

```

Fig. 2.20 Adding a row to data frame using `rbind()`

We use the `rbind()` function to create a new data frame. This mini data frame is called 'Karan'. R throws an error if `rbind()` is used now to bind 'data' and 'Karan' together. The definition of 'Karan' needs to be improved to make the merge successful.

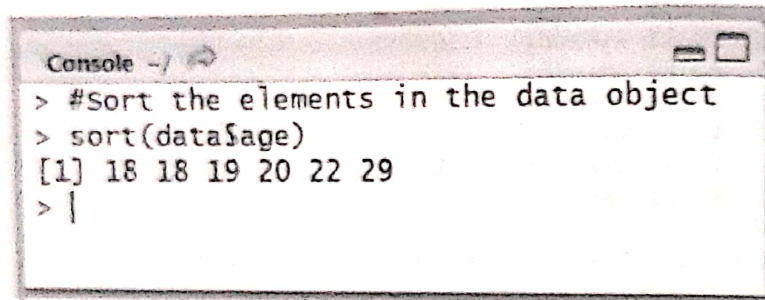
2.5 SORTING DATA FRAMES

To sort the data frame by age, such that the youngest person is on top, and the oldest at the bottom, the `sort()` function can be used. This function sorts a vector into ascending or descending order.

Example 2.4 Write the command in R console to display the values of *age* of *data* in ascending order using the `sort()` function.

Solution:

Here, in Fig. 2.21, it can be observed that if the sorting is done based on the age column, then an ordered version of the age column is returned.



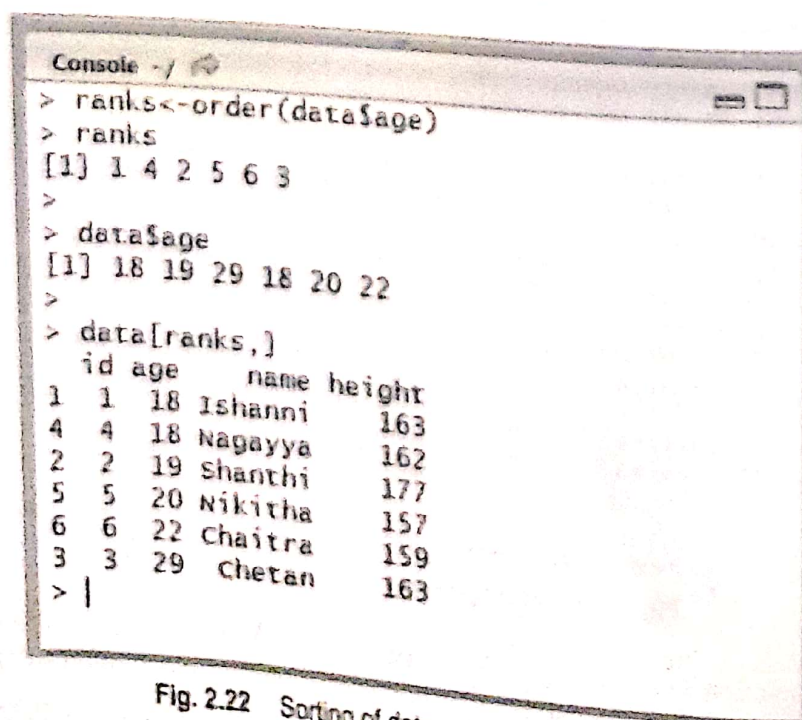
```

Console - / 
> #Sort the elements in the data object
> sort(data$age)
[1] 18 18 19 20 22 29
> |
  
```

Fig. 2.21 Ordered version of age column

To change the ordering of the rows in the data frame, `order()` function can be used instead of the `sort()` function.

As shown in Fig. 2.22, `order()` returns a vector *rank*s with the ranked position of each element. To understand what this means, print the age column and try to reproduce the result. The lowest value in the age column is 18, which is available in both the first and fourth positions. These indices, 1 and 4, thus come first in a *rank*s vector. The second lowest value in the age column is 19, which is available in the second position. The index 2, thus comes next, and so on. The highest value is 29 at the third position of the age column and comes in the last place in *rank*s. The *rank*s vector contains indices that are now used to perform a selection in a scrambled order, such that the result is an ordered data frame.



```

Console - / 
> ranks<-order(data$age)
> ranks
[1] 1 4 2 5 6 3
> 
> data$age
[1] 18 19 29 18 20 22
> 
> data[ranks,]
  id age  name height
1  1  18 Ishanni   163
4  4  18 Nagayya   162
2  2  19 Shanthi   177
5  5  20 Nikitha   157
6  6  22 Chaitra   159
3  3  29 Chetan    163
> |
  
```

Fig. 2.22 Sorting of data using `order()` function

To sort in descending order, the *decreasing* argument is set to TRUE as shown in Fig. 2.23.

```

Console ~/
> data[order(data$age, decreasing = TRUE),]
  id age  name height
3  3  29  Chetan   163
6  6  22 Chaitra   159
5  5  20 Nikitha   157
2  2  19 Shanthi   177
1  1  18 Ishanni   163
4  4  18 Nagayya   162
> |
  
```

Fig. 2.23 Sorting the data in descending order

Programming Questions

- 2.1 Create a character vector of 10 elements with sizes small, medium, and large.
- 2.2 Convert the vector to a factor and place it in an object called sizes.
- 2.3 Identify the class of object sizes.
- 2.4 Display the content of object sizes.
- 2.5 Use the *nlevels* function to see how many levels are there in sizes and put them in a table using the *table* function.
- 2.6 Use the *plot* function to plot the factors.
- 2.7 Use ordered attribute to compare the users categories.
- 2.8 Check if the first element of the factor is less than the second element of the factor.
- 2.9 Create a character vector (*gender_vector*) based on the observation of 7 people.
 - (a) 1st observation – Gender = Male as 'M'
 - (b) 2nd observation – Gender = Female as 'F'
 - (c) 3rd observation – Gender = Female as 'F'
 - (d) 4th observation – Gender = Male as 'M'
 - (e) 5th observation – Gender = Male as 'M'
 - (f) 6th observation – Gender = Male 'M'
 - (g) 7th observation – Gender = Female as 'F'
- 2.10 Convert the character vector *gender_vector* to a factor and assign the result to *factor_gender_vector*.
 - (a) Specify the levels of *factor_gender_vector* and print the result.
 - (b) Ask the *summary()* of *gender_vector* and *factor_gender_vector*. Interpret the results of both vectors.
 - (c) Create two variables *male* <- *factor_gender_vector*[1] and *female* <- *factor_gender_vector*[2] check male larger than female?
 - (d) Specify the levels of *factor_gender_vector* and print the result.
 - (e) Print the *summary()* of *gender_vector* and *factor_gender_vector*. Interpret the results of both vectors.
- 2.11 Using the data set demo (demo.csv), perform the following operations:
 - (a) Retrieve the data set to a variable *emp_demo*.
 - (b) Assign the first 25 entries from *emp_demo*[*gender*] to *emp_gender*.
 - (c) What is the class of *emp_gender*?
 - (d) What is the summary of *emp_gender*?
 - (e) Specify the levels of *emp_gender* and print the result.
 - (f) Assign the first 30 entries from *emp_demo*[*empcat*] to *empcat_vector*.
 - (g) What is the class of *empcat_vector*?
 - (h) From *empcat_vector*, create an ordered factor vector *factor_empcat_vector*, and set the levels accordingly (in ascending order). Generate the summary of *factor_empcat_vector*.
 - (i) Check if the category of the 20th employee is less than that of the 25th employee.